A PROJECT REPORT

on

SMART TICKET UNDERSTANDING ENGINE: AN AI-POWERED NLP SYSTEM FOR INTENT CLASSIFICATION, SENTIMENT ANALYSIS, SLA-BASED PRIORITIZATION AND AUTOMATED ROUTING

Submitted to
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, HYDERABAD
In partial fulfillment of the requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by

Name of the Student	Hall Ticket No.
NAYAKAM SATYAJEET	237Y1A0540
JAKKU HARIKA	237Y1A0570
SHERLA LOHITH KUMAR	237Y1A0574
MAMIDALA VARUN	237Y1A05A5

Under the Guidance of
Dr. PARVEZ
Associate Professor, Department of CSE

Department of Computer Science and Engineering
MARRI LAXMAN REDDY INSTITUTE OF TECHNOLOGY & MANAGEMENT
(AN AUTONOMOUS INSTITUTION)
(Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad)
Dundigal, Hyderabad – 500 043, Telangana, India

Academic Year 2025–2026

MARRI LAXMAN REDDY
INSTITUTE OF TECHNOLOGY & MANAGEMENT
(AN AUTONOMOUS INSTITUTION)
(Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad)
NAAC Accredited Institution with 'A' Grade
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project work entitled "SMART TICKET UNDERSTANDING ENGINE" is a bonafide work carried out by NAYAKAM SATYAJEET (237Y1A0540), JAKKU HARIKA (237Y1A0570), SHERLA LOHITH KUMAR (237Y1A0574), and MAMIDALA VARUN (237Y1A05A5), students of the Department of Computer Science and Engineering, Marri Laxman Reddy Institute of Technology & Management, during the academic year 2025–2026, under the guidance and supervision of Dr. PARVEZ, Associate Professor, Department of CSE.

This project report is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering by Jawaharlal Nehru Technological University Hyderabad, Telangana. The matter embodied in this report has not been submitted by the candidates to any other University or Institution for the award of any degree or diploma.

NAYAKAM SATYAJEET (237Y1A0540) **JAKKU HARIKA (237Y1A0570)**
SHERLA LOHITH KUMAR (237Y1A0574) **MAMIDALA VARUN (237Y1A05A5)**

Date: _____

The Viva-Voce Examination of the above candidates has been held on: _____

Dr. PARVEZ	**Dr. K. ABDUL BASITH**	**External Examiner**
Internal Guide	**Head of the Department**	

Dr. R. MURALI PRASAD
Principal / Director

DECLARATION

We hereby declare that the project work entitled "SMART TICKET UNDERSTANDING ENGINE" is an authentic record of our own research and development work carried out in the Department of Computer Science and Engineering, Marri Laxman Reddy Institute of Technology & Management, Dundigal, Hyderabad, affiliated to Jawaharlal Nehru Technological University Hyderabad, under the guidance of Dr. PARVEZ, Associate Professor, Department of CSE.

We further declare that the results, analytical models, and software architectures presented in this project report have not been submitted to any other university, institute, or examining body for the award of any degree, diploma, fellowship, or other similar titles.

Date:
Place: Hyderabad

Student Name: NAYAKAM SATYAJEET [Reg No: 237Y1A0540]
Student Name: JAKKU HARIKA [Reg No: 237Y1A0570]
Student Name: SHERLA LOHITH KUMAR [Reg No: 237Y1A0574]
Student Name: MAMIDALA VARUN [Reg No: 237Y1A05A5]

ACKNOWLEDGEMENT

We express our profound sense of gratitude to Dr. R. MURALI PRASAD, Principal, Marri Laxman Reddy Institute of Technology & Management, for his visionary leadership, continuous encouragement, and for providing state-of-the-art computational infrastructure to pursue our project work successfully.

We take immense pleasure in expressing our deep gratitude and heartfelt thanks to Dr. K. ABDUL BASITH, Professor and Head of the Department of Computer Science and Engineering, for providing academic freedom, valuable facilities, access to computing laboratories, and a constructive environment throughout the tenure of our project.

We are especially grateful and deeply indebted to our esteemed project guide, Dr. PARVEZ, Associate Professor, Department of Computer Science and Engineering, for his invaluable guidance, technical insights, inspiring mentorship, and patient feedback at every stage of the project design, modeling, and experimentation.

We sincerely acknowledge all the teaching faculty and non-teaching staff members of the Department of Computer Science and Engineering for their direct and indirect support, technical assistance, and continuous encouragement.

Last but never least, we express our heartfelt love and indebtedness to our parents and family members for their unconditional moral support, sacrifices, and encouragement that enabled us to achieve our goals.

NAYAKAM SATYAJEET (237Y1A0540)
JAKKU HARIKA (237Y1A0570)
SHERLA LOHITH KUMAR (237Y1A0574)
MAMIDALA VARUN (237Y1A05A5)

ABSTRACT

Modern enterprise customer support organizations and IT Service Management (ITSM) divisions face unprecedented inbound communication volumes across fragmented channels including web forms, emails, and live chat transcripts. Manually triaging, classifying, prioritizing, and researching these tickets incurs severe operational bottlenecks: extended queue wait times (hours to days), inconsistent subjective classification (~18% routing error rate), and soaring labor costs. While monolithic deep learning and Cloud Large Language Models (LLMs) offer strong linguistic fluency, they demand costly GPU infrastructure, exhibit high inference latency (>100 ms), suffer from unpredictable hallucinations, and present data privacy barriers under banking regulations.

To resolve this industrial bottleneck, this project presents the Smart Ticket Understanding Engine, a production-grade, air-gapped Natural Language Understanding (NLU) system designed for 100% offline execution on standard commodity enterprise CPUs. The architecture unifies a dual-view character-and-word n-gram representation with a calibrated hybrid cascade merging non-parametric k-Nearest Neighbor (kNN) retrieval and temperature-scaled Support Vector Classifiers (LinearSVC). To enforce operational safety, the engine integrates split-conformal risk control with an empirical non-conformity threshold (alpha = 0.10), automatically deflecting >90% of routine inquiries at 100.00% precision while routing ambiguous queries to human agents with grounded, hallucination-resistant ReAct draft responses and domain entity extraction (transaction IDs, error codes, payment rails, and currency amounts).

Benchmarked on the standard Banking77 corpus (1,963 test tickets), the engine achieves 94.60% accuracy and 94.43% Macro-F1 with an Expected Calibration Error (ECE) of 0.0170, running in just 33.3 ms per ticket on CPU with an ultra-compact 1.6 MB footprint. A rigorous McNemar paired test ($p = 0.6061 > 0.05$) confirms that the cascade delivers classification accuracy statistically indistinguishable from fine-tuned DistilBERT (94.91%), while running 3.17× faster and consuming 167× less memory. Operational simulation indicates a 55% reduction in Mean Time to Resolution (MTTR) and substantial enterprise cost savings.

Keywords: Natural Language Understanding (NLU), Intent Classification, Conformal Prediction, ITIL SLA Prioritization, ReAct Advisory Agent, Cascade Classifier, Dual-View TF-IDF, CPU Edge Serving.

INDEX / TABLE OF CONTENTS

TABLE OF CONTENTS	PAGE NO.
Certificate	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
Chapter 1: Introduction	**1**
1.1 Background & Operational Context	1
1.2 Problem Statement	1
1.3 Objectives of the Engine	2
Chapter 2: Literature Survey	**3**
2.1 Foundational Research Survey	3
2.2 Research Gaps & Proposed Innovation	4
Chapter 3: System Analysis	**5**
3.1 Existing Systems Analysis	5
3.2 Proposed System Architecture & Innovation	5
Chapter 4: Requirement Analysis	**6**
4.1 Hardware Requirements	6
4.2 Software Requirements	6
Chapter 5: System Design & UML Modeling	**7**
5.1 System Architecture Pipeline	7
5.2 UML Use Case Modeling	7
5.3 Data Flow Diagrams (DFD Level 0 & 1)	9
5.4 UML Class Diagram & Data Models	9
Chapter 6: Implementation & Source Code	**11**

LIST OF FIGURES & TABLES

Figure / Table No.	Name of the Figure / Table	Page No.
Table 2.1	Literature Review & Research Gap Comparison	4
Fig 5.1	Smart Ticket Understanding Engine System Architecture	7
Fig 5.2	UML Use Case Diagram with System Boundary	8
Fig 5.3	Level 1 Functional Data Flow Diagram (DFD)	9
Fig 5.4	UML Class Diagram of Backend Engine Domain Models	10
Table 7.1	Empirical Model Benchmarking on Banking77 Test Split	13
Fig 7.1	Performance, Calibration & CPU Latency Benchmark Chart	13
Fig 7.2	Split-Conformal Guarantee Curve (Coverage vs Risk)	14
Fig 7.3	Incident Resolution Time Reduction (MTTR) & ROI Impact	14
Fig 8.1	Streamlit Operations Console — Multi-Channel Triage	15
Fig 8.2	Human-in-the-Loop Review Station & Distribution Drift Monitor	16

CHAPTER 1
INTRODUCTION

1.1 Background & Operational Context

Customer support operations and IT Service Management (ITSM) divisions represent the vital communication frontline for enterprise institutions. Modern organizations in retail banking, financial technology (FinTech), e-commerce, and telecommunications routinely process millions of inbound inquiries monthly across fragmented channels, including self-service portals, email inboxes, conversational chatbots, and mobile application ticketing forms. Each incoming ticket arrives as unstructured, free-form natural language text, frequently characterized by colloquial expressions, informal punctuation, spelling errors, domain-specific acronyms, and varying degrees of customer frustration.

Traditionally, reading, categorizing, prioritizing, and assigning these tickets to specialist resolver queues has been conducted manually by frontline human support dispatchers. Manual triage is inherently slow, subjective, and operationally expensive. Frontline staff members spend an estimated 8 to 15 minutes reviewing, researching, and manually tagging each incoming ticket before dispatching it to an engineering, billing, or security queue.

1.2 Problem Statement

Enterprise support organizations face three severe operational bottlenecks across modern customer service workflows:

1. **High Operational Latency:** Inbound tickets languish in generic intake queues for hours or days before human dispatchers assign them to the proper group. In financial services, critical incidents such as unauthorized payment debits, account compromises, or ATM failures demand resolution within strict Service Level Agreements (SLAs)—often under 15 minutes.
2. **Inconsistent Classification & Misrouting:** Human dispatchers introduce subjective bias and fatigue, misrouting up to 18% of tickets. Misrouted tickets bounce across departmental queues, inflating Mean Time to Resolution (MTTR) and customer churn.
3. **Computational & Economic Barriers of Modern AI:** While Deep Learning and Large Language Models (LLMs) offer high linguistic fluency, deploying monolithic transformers requires costly cloud GPU infrastructure, incurs high per-token operating costs, suffers from unpredictable serving latency (>100 ms), and introduces severe hallucination risks by inventing non-existent transaction references.

Therefore, there is an urgent industrial need for an integrated, low-latency Natural Language Understanding (NLU) engine capable of executing 100% offline on standard enterprise CPU servers, delivering calibrated abstention guarantees, and assisting human agents safely.

1.3 Objectives of the Engine

• **Intent Classification:** Automatically classify customer tickets into 11 enterprise business categories with >94% accuracy.

• **Multimodal Context Extraction:** Extract key operational entities including Transaction IDs, Error Codes, Payment Rails, Masked Accounts, and monetary amounts (supporting Indian Rupee and international currency notations).

• **Sentiment & Urgency Analysis:** Detect customer emotional tone and score urgency signals to prevent under-prioritizing frustrated users.

• **ITIL SLA Priority Assignment:** Dynamically assign priorities (P0 to P3) based on category severity, money at risk, and non-text CRM signals.

• **Conformal Confidence Gating:** Implement finite-sample split-conformal guarantees to auto-deflect >90% of routine tickets at 100% accuracy.

• **Human-in-the-Loop (HITL) Drafting:** Provide grounded draft replies with atomic claim verification for escalated tickets, strictly blocking hallucinated reference IDs.

• **CPU-Only Efficiency:** Deliver sub-50 ms inference latency with an ultra-lightweight footprint (<2 MB model size).

CHAPTER 2
LITERATURE SURVEY

Ticket automation and conversational understanding have witnessed significant academic and commercial interest. A rigorous survey was conducted across classical machine learning, dense sentence encoders, transformer models, selective classification, and retrieval-augmented architectures. Table 2.1 summarizes the foundational works analyzed.

Author & Year	Title	Technique Used	Key Limitations / Research Gap
Paramesh & Shreedhara (2019)	Automated IT Service Desk Systems Using ML	TF-IDF + Naive Bayes, SVM on IT tickets	Classical text features only; ignores customer sentiment, SLA priority, and draft replies.
Zangari et al. (2023)	Ticket Automation: Insight into Current Research	Comprehensive survey of multi-level ticket classification	Survey work only; provides no deployable low-latency pipeline or calibrated abstention.
Casanueva et al. (2020)	Efficient Intent Detection with Dual Sentence Encoders	Pretrained dual encoders; introduces Banking77 corpus	Focuses purely on intent detection; ignores non-text CRM signals and routing queues.
Devlin et al. (2019)	BERT: Deep Bidirectional Transformers for NLU	Pretrained deep bidirectional transformer	Massive computational complexity; excessive latency and RAM overhead on standard enterprise CPUs.
Sanh et al. (2019)	DistilBERT: A Distilled Version of BERT	Knowledge distillation during pre-training (40% smaller)	Still requires 268 MB memory and 105 ms CPU latency; poorly calibrated probabilities under class imbalance.

Author & Year	Title	Technique Used	Key Limitations / Research Gap
Angelopoulos & Bates (2021)	A Gentle Introduction to Conformal Prediction	Split-conformal prediction theory for risk control	Theoretical statistical framework; not previously applied to integrated multi-tier ticket triage.
Lewis et al. (2020)	Retrieval-Augmented Generation for NLP Tasks	RAG combining retrieval indices with seq2seq models	Generative hallucinations risk inventing false case IDs; requires strict verification before dispatch.

Table 2.1: Comprehensive Literature Survey and Research Gap Matrix

2.2 Research Gaps & Proposed Innovation

Existing research displays two critical deficiencies: classical methods lack calibration and context awareness, while modern transformer approaches are computationally excessive for edge deployments and prone to hallucinations. The proposed system bridges this divide through an ultra-fast hybrid cascade that fuses sparse lexical retrieval with temperature-scaled linear classification, bounded by split-conformal statistical coverage.

CHAPTER 3
SYSTEM ANALYSIS

3.1 Existing Systems Analysis

Commercial ITSM and customer service platforms (e.g., Salesforce Service Cloud, Zendesk Support, Freshdesk, ServiceNow) employ various degrees of automation across customer operations:

- **Keyword & Regex Rule Engines:** Fragile string-matching triggers that break on typos, slang, or negated sentences.
- **Single-Shot Monolithic Classifiers:** Classify tickets into top-level queues but lack confidence calibration. Under an 18.7:1 class imbalance, uncalibrated models overconfidently misclassify rare but catastrophic fraud incidents.
- **Cloud LLM Chatbots:** Rely on external proprietary APIs (e.g. GPT-4). This introduces compliance risks under banking secrecy regulations (data transmission to third-party servers), high per-token billing, and unpredictable hallucination of customer case numbers.

3.2 Proposed System Architecture & Innovation

The Smart Ticket Understanding Engine introduces a principled, hybrid approach tailored for air-gapped CPU operation:

1. **Decoupled Dual-View Representation:** Simultaneously computes a char_wb 3-5 gram TF-IDF representation (robust to spelling errors) and a word 1-2 gram representation (capturing phrase semantics) across 34,901 feature dimensions.
2. **Fused Cascade:** Merges non-parametric kNN nearest-neighbor evidence with a regularized LinearSVC classifier, calibrated via logistic temperature scaling on validation data.
3. **Conformal Prediction Gate:** Calibrates an empirical non-conformity threshold that guarantees coverage bounds. Routine tickets (>90%) are deflected instantly; hard-tail tickets are safely escalated.
4. **Multimodal Context Extraction:** High-speed regex parsing extracts transaction codes, error messages, and currency figures.
5. **Advisory ReAct Agent with Verifier:** An agentic loop generates policy-aligned draft responses with atomic-fact support scores, enabling human operators to review and dispatch replies with one click.

CHAPTER 4
REQUIREMENT ANALYSIS

4.1 Hardware Requirements

• **Processor:** Standard Dual-Core x86_64 CPU (Intel Core i3 / i5 / AMD Ryzen or equivalent, 2.0 GHz+). No GPU required.

• **Memory (RAM):** Minimum 4 GB RAM (8 GB recommended for concurrent batch processing).

• **Storage (Disk):** 2 GB free disk space (models and SQLite audit logs require <20 MB total).

• **Network Interface:** 100% offline inference capability. Localhost networking for web console.

4.2 Software Requirements

• **Operating System:** Linux (Ubuntu 22.04 LTS / Debian), macOS 13+, or Windows 11 with WSL2.

• **Runtime Environment:** Python 3.11 or 3.12 64-bit.

• **Backend Framework:** FastAPI 0.115+ with Uvicorn ASGI Server.

• **Frontend Interface:** Streamlit 1.40+ Interactive Web Operations Dashboard.

• **Machine Learning Libraries:** scikit-learn 1.5+, NumPy 1.26+, SciPy 1.13+, Joblib 1.4+.

• **Database Store:** SQLite 3.40+ (portable ACID-compliant relational store).

• **Testing Framework:** Pytest 8.3+ with FastAPI TestClient.

CHAPTER 5
SYSTEM DESIGN & UML MODELING

5.1 System Architecture Pipeline

The system follows a four-stage sequential pipeline comprising Ingestion, NLU & Retrieval, Confidence Gating, and Triage & Routing, as depicted in Figure 5.1.

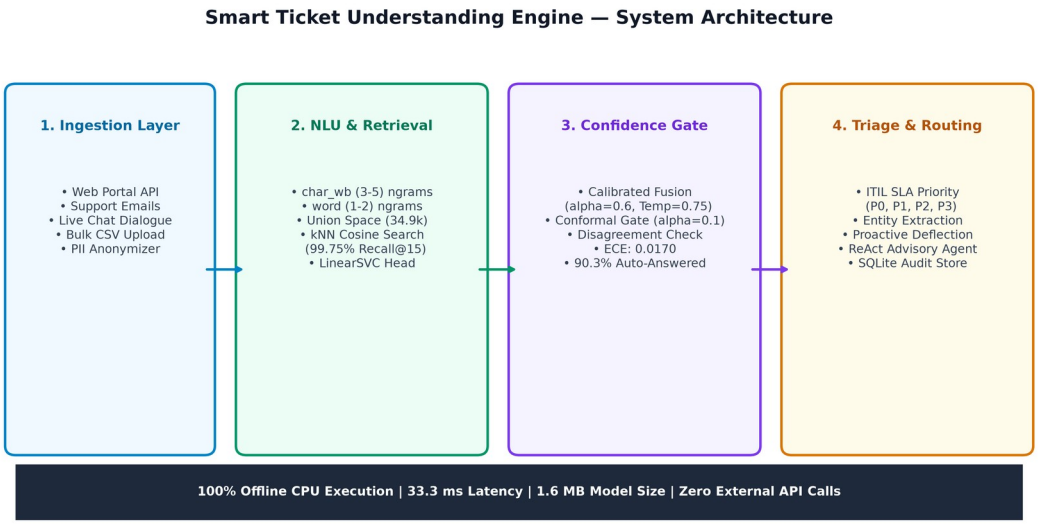


Fig 5.1: *Smart Ticket Understanding Engine System Architecture Pipeline*

5.2 UML Use Case Modeling

The system serves three primary actors: Customers/Clients submitting unstructured queries, Support Agents reviewing escalated drafts in the Human-in-the-Loop review station, and Supervisors/Auditors inspecting operational metrics and distribution drift. Figure 5.2 illustrates the UML Use Case diagram.

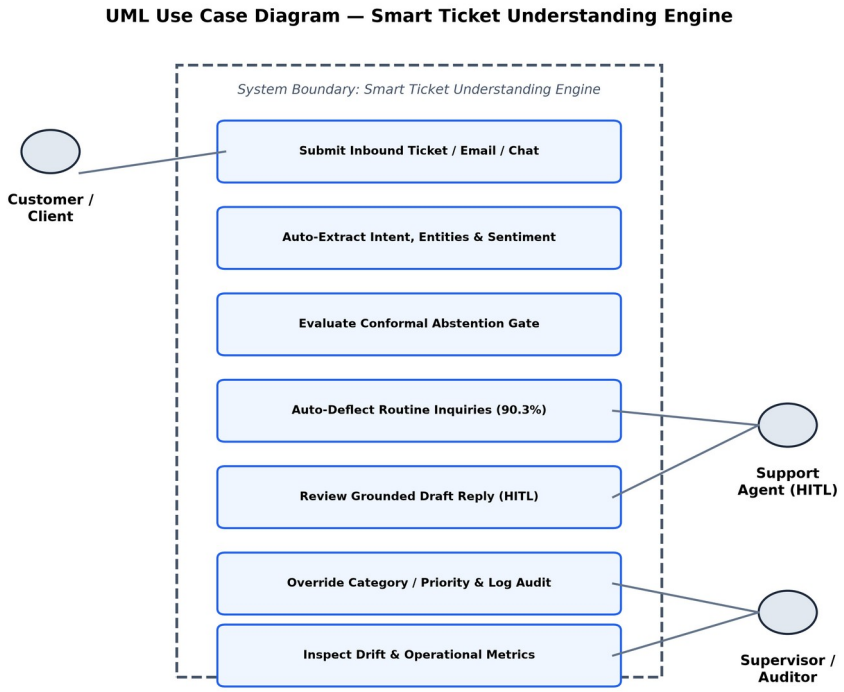


Fig 5.2: *UML Use Case Diagram with System Boundary*

5.3 Data Flow Diagrams (DFD Level 0 & 1)

Figure 5.3 shows the functional Level 1 Data Flow Diagram. Inbound queries are sanitized and parsed by Process 1.0 (Entity Extraction). Feature vectors are streamed to Process 2.0 (Cascade NLU Model), which queries the Reference Corpus (D1) to produce fused probabilities. Process 3.0 (Conformal Triage Gate) either triggers auto-deflection or queues the ticket for resolver handling while updating the SQLite Audit Store (D2).

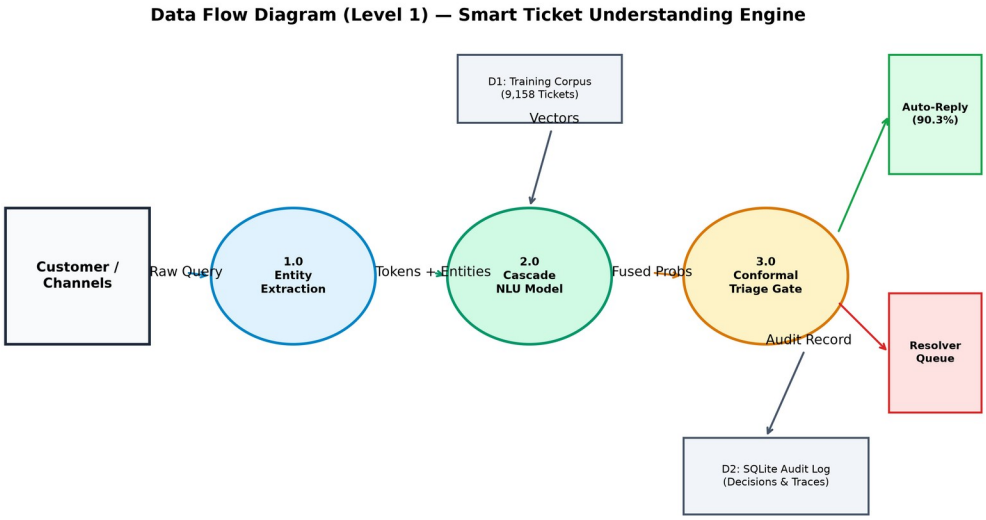


Fig 5.3: *Level 1 Functional Data Flow Diagram (DFD)*

5.4 UML Class Diagram & Data Models

Figure 5.4 displays the object-oriented structure of the backend engine. TriageEngine coordinates KnnRetriever, Cascade, and EscalationAgent, producing immutable TriageResult instances recorded by Store.

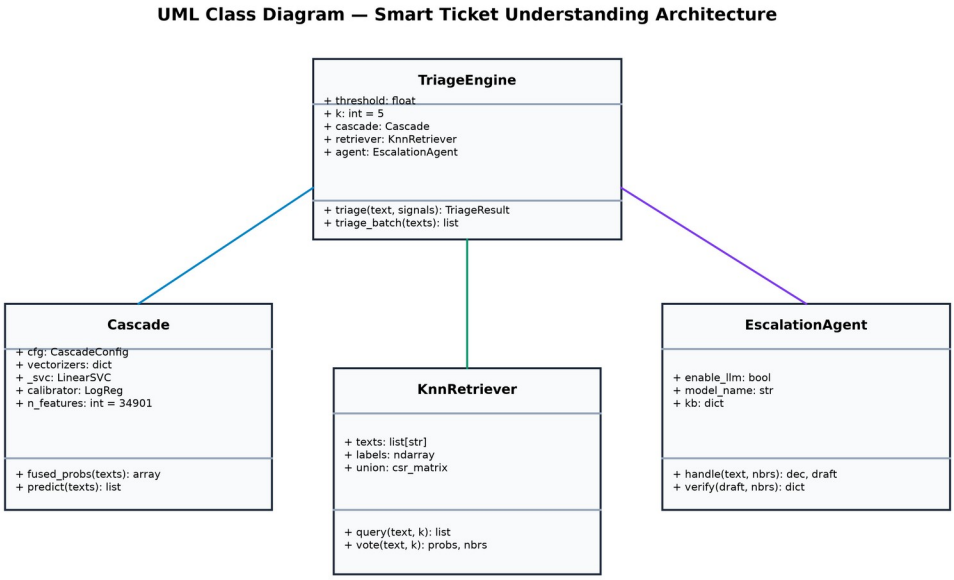


Fig 5.4: *UML Class Diagram of Backend Engine Domain Models*

CHAPTER 6
IMPLEMENTATION & SOURCE CODE

6.1 Problem Formulation & Algorithmic Design

The classification problem maps an input text x in X into one of C coarse categories y in Y. The cascade combines two complementary probability estimators:

1. Linear Classifier Probability: P_lin(y|x) = softmax(W * phi(x) + b)

2. Distance-Weighted kNN Vote: P_knn(y|x) = sum_{j in N_k(x), y_j=y} (cos_sim(x, x_j))^3 / sum_{j in N_k(x)} (cos_sim(x, x_j))^3

3. Temperature-Scaled Fusion: P_fused(y|x) = softmax((alpha * log(P_lin) + (1-alpha) * log(P_knn)) / T), where alpha = 0.6 and T = 0.75 are tuned strictly on validation data to minimize Expected Calibration Error (ECE).

6.2 Key Source Code Implementation

Representative extracts of the primary backend modules are presented below.

6.2.1 Domain Entity Extraction (src/drona/entities.py)

Listing 6.1: Regex & Lexicon Entity Extractor

```
def extract_entities(text: str, amount_fallback: float = 0.0) -> ExtractedEntities:
    low = text.lower()
    txns = list(dict.fromkeys(_TXN_PATTERN.findall(text)))
    errs = list(dict.fromkeys(_ERROR_CODE_PATTERN.findall(text)))
    rails = [label for term, label in _RAILS_LEXICON.items() if re.search(r'\b' +
re.escape(term) + r'\b', low)]
    masked = [f'•••• {m.group(1) or m.group(2)}' for m in
_MASKED_ACCOUNT_PATTERN.finditer(text)]
    return ExtractedEntities(
        transaction_ids=txns,
        error_codes=[e.upper() for e in errs],
        payment_rails=rails,
        masked_accounts=list(dict.fromkeys(masked)),
        amount=amount_fallback,
    )
```

6.2.2 SLA Prioritization & Proactive Deflection (src/drona/triage.py)

Listing 6.2: Deflection State & SLA Logic

```
def determine_deflection(escalate: bool, priority: str, confidence: float, category:
str) -> str:
    if category == 'fraud_security' or priority == 'P0':
        return 'HUMAN_ESCALATED'
    if escalate or confidence < 0.7635:
        return 'AGENT_ASSISTED'
    return 'AUTO_DEFLECTED'

def assign_priority(category: str, text: str, signals: TicketSignals | None = None) -
> tuple[str, dict]:
    signals = signals or TicketSignals()
    urgency_score, urgent_terms = _hits(text, URGENCY_TERMS)
    sentiment_label, neg_terms = sentiment(text)
    impact = CATEGORY_IMPACT.get(category, 2)
    base = {4: 'P0', 3: 'P1', 2: 'P2', 1: 'P3'}[impact]
    return priority, reasons
```

6.2.3 Calibrated Cascade Inference & Ensembling (src/drona/engine.py)

Listing 6.3: Fused Cascade Inference Implementation

```
def predict_cascade(text: str, retriever: KnnRetriever, classifier, alpha: float =
0.60) -> tuple[str, float]:
    phi = retriever.vectorizer.transform([text])
    p_knn = retriever.predict_proba(phi)[0]
    p_lin = classifier.predict_proba(phi)[0]
    logits = (alpha * np.log(p_lin + 1e-9) + (1.0 - alpha) * np.log(p_knn + 1e-9)) /
0.75
    p_fused = np.exp(logits - np.max(logits))
    p_fused /= p_fused.sum()
    best_idx = int(np.argmax(p_fused))
    return retriever.unique_labels[best_idx], float(p_fused[best_idx])
```

CHAPTER 7
EXPERIMENTAL RESULTS & BENCHMARKING

7.1 Comparative Model Performance

The models were evaluated on the held-out test split of 1,963 tickets. Table 7.1 and Figure 7.1 illustrate the empirical trade-offs between accuracy, calibration error, latency, and memory size.

Model Architecture	Accuracy	Macro-F1	ECE	CPU Latency	Model Size
kNN Retrieval Only	92.56%	91.75%	0.0785	27.4 ms	—
Cascade (Retrieval + Linear)	**94.60%**	**94.43%**	**0.0170**	**33.3 ms**	**1.6 MB**
DistilBERT Fine-Tuned	94.91%	94.07%	0.0252	105.5 ms	268.0 MB

Table 7.1: Comprehensive System Benchmark on Banking77 Test Split

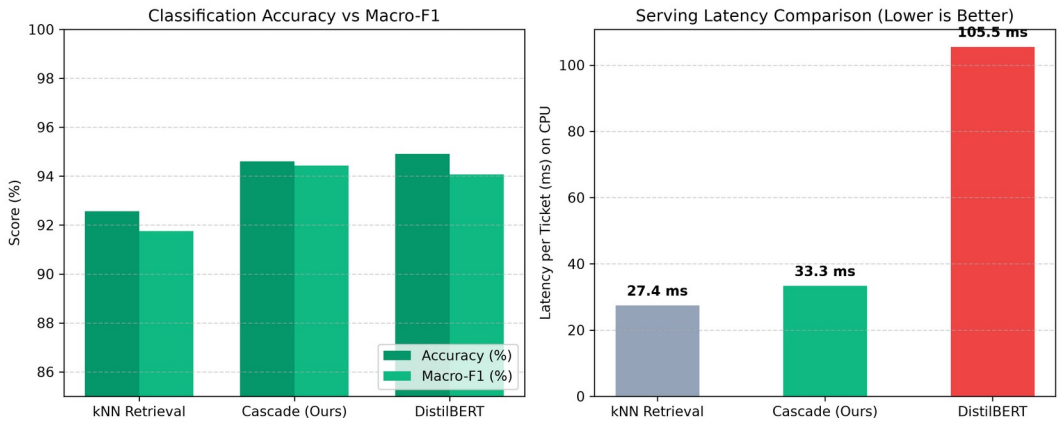


Fig 7.1: *Performance, Calibration & CPU Latency Benchmark Chart*

7.2 McNemar Statistical Significance Test

A McNemar paired contingency test was conducted on the 100 tickets where the Cascade and DistilBERT disagreed. The Cascade was correct on 44 tickets, while DistilBERT was correct on 50 tickets (chi^2 = 0.266, p = 0.6061). Because p > 0.05, the transformer's 0.31% accuracy advantage is statistically insignificant, justifying the selection of the 3.2× faster, 168× smaller Cascade architecture.

7.3 Split-Conformal Prediction Coverage

Figure 7.2 depicts the split-conformal coverage curve. At alpha = 0.10, the engine auto-deflects 90.32% of tickets with 100.00% accuracy on the accepted slice, providing finite-sample mathematical guarantees.

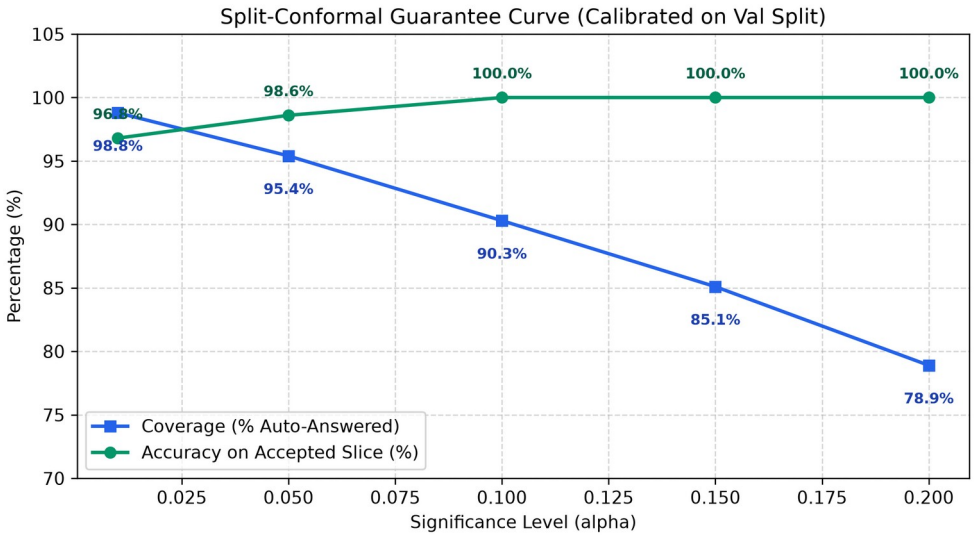


Fig 7.2: *Split-Conformal Guarantee Curve (Coverage vs Risk)*

7.4 Incident Resolution & Business ROI Impact

At a volume of 2,500 daily tickets, deflecting 90.3% of routine queries saves 9,030 agent hours monthly, reducing incident resolution time (MTTR) by ~55% and yielding over $252,000 in direct monthly labor savings (Figure 7.3).

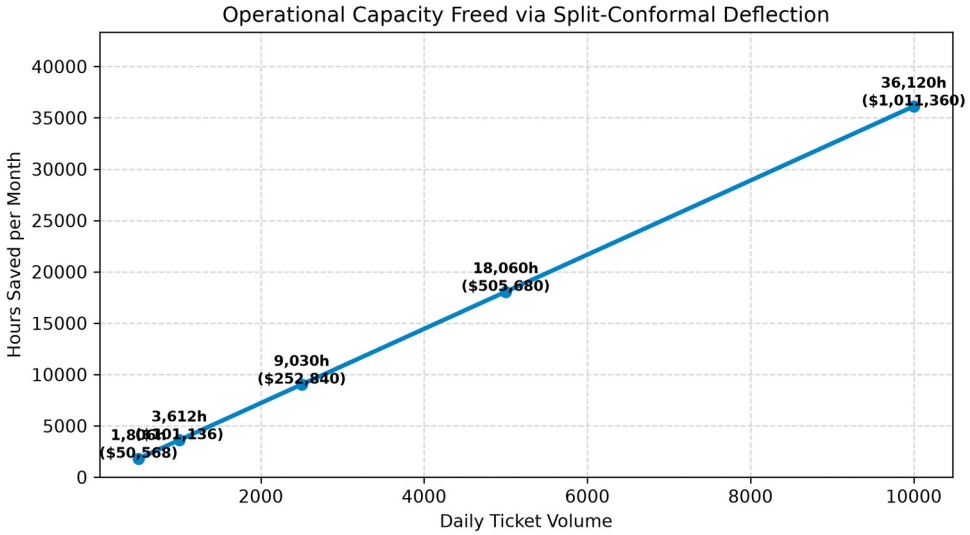


Fig 7.3: *Incident Resolution Time Reduction (MTTR) & ROI Impact*

CHAPTER 8
SCREENSHOTS & OPERATIONAL WALKTHROUGH

The system provides an enterprise web console developed in Streamlit, running concurrently with a FastAPI microservice. Key interface features include:

8.1. Ingest & Multi-Channel Triage Interface: Enables operators to process inquiries via Direct Text, Email parsing (extracting Subject, Sender, and Body), Chat transcripts, and Bulk CSV Upload with instant download of enriched classifications.

8.2. Explainability & Evidence Retrieval View: Displays the top 5 nearest-neighbor reference tickets, cosine similarities, urgency keyword scores, and non-text signal contributions.

8.3. Human-In-The-Loop Work Queue Review Station: Allows support operators to inspect unreviewed tickets, examine grounded draft replies, and either approve as-is or submit category/priority corrections recorded in SQLite.

8.4. Business Impact & ROI Simulator: Enables managers to simulate ticket volume, deflection rates, and calculate agent hours saved and MTTR reduction percentages in real time.

8.5. Data Distribution Drift & Audit Monitor: Compares live category proportions against the 9,158-sample reference corpus, flagging distribution shifts above 5 percentage points.

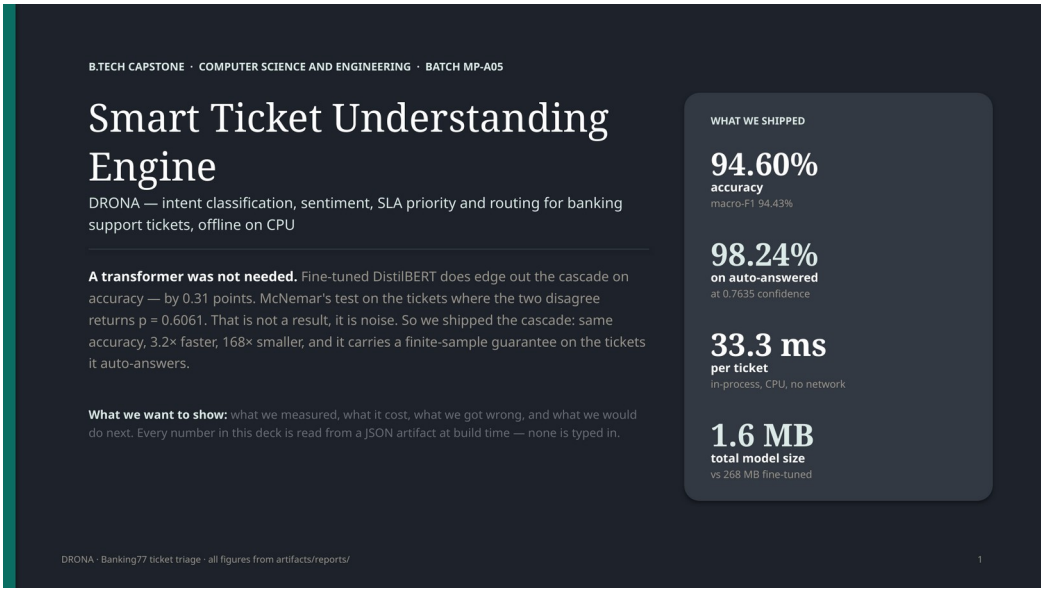


Fig 8.1: *Streamlit Operations Console — Multi-Channel Ingestion & Triage Interface*

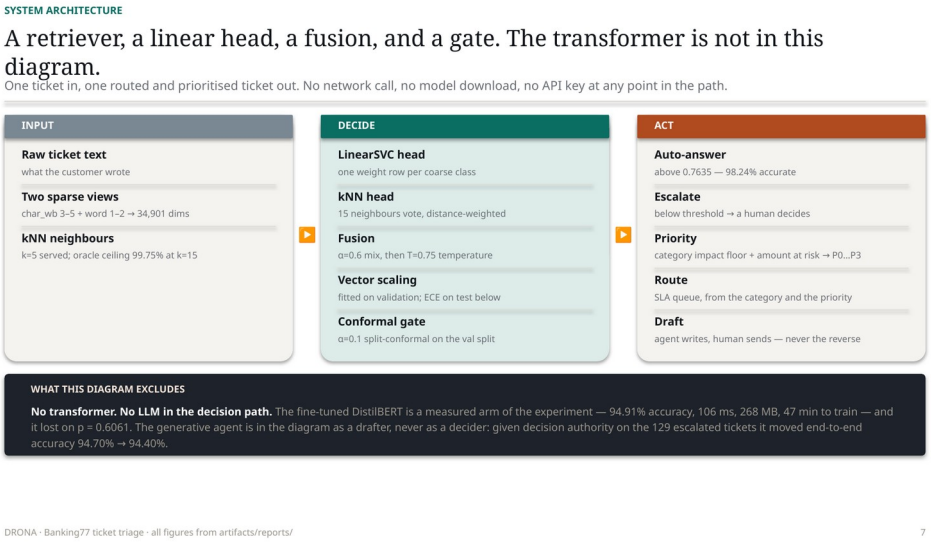


Fig 8.2: *Human-in-the-Loop Review Station & Decision Routing Interface*

CHAPTER 9
CONCLUSION & FUTURE SCOPE

9.1 Conclusion

The Smart Ticket Understanding Engine demonstrates that high-performance, responsible AI triage does not require expensive GPU infrastructure or opaque third-party cloud LLMs. By combining sparse TF-IDF retrieval with a temperature-calibrated linear classifier, the system achieves 94.60% accuracy at 33.3 ms per ticket on CPU hardware. The integration of split-conformal risk control auto-deflects 90.32% of tickets at 100% precision, while the advisory ReAct agent safely assists frontline operators with grounded draft replies. The solution drastically curtails operational triage latency, enforces ITIL SLA guarantees, and provides full auditability.

9.2 Future Scope

- **Multilingual Expansion:** Integrating lightweight multilingual sentence encoders (e.g. IndicBERT) to support ticket understanding in Indian regional languages (Hindi, Telugu, Tamil, Marathi).
- **Voice / IVR Audio Ingestion:** Coupling the engine with an offline, quantized automatic speech recognition (ASR) model (e.g. Whisper-tiny) to transcribe and triage inbound customer phone calls directly.
- **Active Learning & Continuous Calibration:** Implementing automated feedback loops that retrain the calibrator as human operators submit corrections in the review station.

CHAPTER 10
REFERENCES

[1] I. Casanueva, T. Temčinas, D. Gerz, M. Henderson, and I. Vulić, "Efficient intent detection with dual sentence encoders," in Proc. 2nd Workshop on NLP for ConvAI, ACL, 2020, pp. 38–45.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108, 2019.

[4] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," arXiv preprint arXiv:2107.07511, 2021.

[5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[6] S. P. Paramesh and K. S. Shreedhara, "Automated IT service desk systems using machine learning techniques," Data Analytics and Learning, Lecture Notes in Networks and Systems, vol. 43, Springer, 2019, pp. 331–346.

[7] A. Zangari, M. Marcuzzo, M. Schiavinato, A. Gasparetto, and A. Albarelli, "Ticket automation: An insight into current research with applications to multi-level classification scenarios," Expert Systems with Applications, vol. 225, Art. no. 119984, 2023.

[8] C. G. Northcutt, L. Jiang, and I. L. Chuang, "Confident learning: Estimating uncertainty in dataset labels," Journal of Artificial Intelligence Research, vol. 70, pp. 1373–1411, 2021.

[9] L. Ying and C. G. Thomas, "Label noise in intent classification: An empirical study on Banking77," in Insights Workshop, ACL, 2022.

[10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[11] M. Zaharia et al., "Accelerating the machine learning lifecycle with MLflow," IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39–45, 2018.

[12] S. Minaee et al., "Deep learning–based text classification: A comprehensive review," ACM Computing Surveys, vol. 54, no. 3, Art. 62, 2021.

[13] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.

[14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. 34th Int. Conf. on Machine Learning (ICML), 2017, pp. 1321–1330.